

Bài tập lý thuyết tuần 6

Họ và tên: Nguyễn Văn Phúc Huy

MSSV: 23110163

Lớp: 23TTH (Sáng thứ 6)

Bài tập:

Khái niệm cơ bản

Tính $n!$ theo phân tích thừa số nguyên tố:

- Nếu xét p là số nguyên tố thì $\mu(p) = \sum_{i=1}^{\infty} [\frac{n}{p^i}]$ là lũy thừa của p trong $n!$.
- Ví dụ: $n = 5 \rightarrow p = 2, 3, 5$

$$\mu(2) = [\frac{5}{2}] + [\frac{5}{2^2}] = 3$$

$$\mu(3) = [\frac{5}{3}] = 1$$

$$\mu(5) = [\frac{5}{5}] = 1$$

- Vậy $n! = 2^3 \cdot 3^1 \cdot 5^1$
- **Bài tập về nhà:** Viết chương trình tính $n!$ theo phương pháp này.

Bài làm

Mã giả cho bài toán phân tích thừa số nguyên tố của $n!$

```
prime_factorization(n):  
    if n < 0:  
        return "Lỗi: n phải là số nguyên dương"  
    if n == 0 hay n == 1:  
        return {}  
  
    primes = tất cả các số nguyên tố nhỏ hơn  
  
    Tạo một dict rỗng tên là prime_factors  
  
    Với mỗi p TRONG primes:  
        count = 0  
        power = p  
  
        Khi power <= n:  
            count = count + Lấy_Phần_Nguyên(n / power)  
            power = power * p  
  
        prime_factors[p] = count  
  
    return prime_factors
```

```
In [5]: def get_all_primes(n):  
    if n < 2:  
        return []  
    primes = []  
    for num in range(2, n + 1):  
        is_prime = True  
        for divisor in range(2, int(num**0.5) + 1):  
            if num % divisor == 0:  
                is_prime = False  
                break  
        if is_prime:  
            primes.append(num)
```

```

        if is_prime:
            primes.append(num)
        return primes

def prime_factorization(n):
    if n < 0:
        return None, "Ko nhận số âm"
    if n == 0 or n == 1:
        return 1, {}

    primes = get_all_primes(n)
    prime_factors = {}

    # phi(p)
    for p in primes:
        count = 0
        power = p
        while power <= n:
            count += n // power
            power *= p
        prime_factors[p] = count

    # n!
    result = 1
    for p, count in prime_factors.items():
        result *= (p ** count)

    return result, prime_factors

n = 5
result, factors = prime_factorization(n)

print(f"Prime factorization of {n}")
for p, count in factors.items():
    print(f"phi({p}) = {count}")

# print factorization result
factor_str = " * ".join([f"{p}^{c}" for p, c in factors.items()])
print(f"\n{n}! = {factor_str}")
print(f"{n}! = {result}")

```

Prime factorization of 5

phi(2) = 3
 phi(3) = 1
 phi(5) = 1

5! = 2³ * 3¹ * 5¹
 5! = 120

Hàm `get_all_primes(n)` trả về danh sách tất cả các số nguyên dương nhỏ hơn hoặc bằng n.

Hàm `prime_factorization(n)` tính n! và phân tích thừa số nguyên tố của n! theo định lý Legendre, sau đó in ra kết quả.

Hoạt động như sau:

1. Kiểm tra nếu $n < 0$, trả về lỗi.
2. Nếu n là 0 hoặc 1, trả về 1 và một dict rỗng.
3. Lấy danh sách các số nguyên tố nhỏ hơn hoặc bằng n.
4. Tính số mũ mu(p) cho từng số nguyên tố p bằng cách sử dụng vòng lặp để tính tổng $n // p^k$.
5. Tính n! bằng cách nhân các thừa số nguyên tố với số mũ tương ứng và trả về kết quả cùng với dict chứa các thừa số nguyên tố và số mũ của chúng.